

02477 Bayesian Machine Learning 2025R - Proposed solution

```
In [3]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn as snb
import jax.numpy as jnp
from jax import random

import numpy as np
from jax.scipy.stats import binom as binom_dist

import jax
jax.config.update("jax_enable_x64", True)

snb.set_style('darkgrid')
```

Part 1: Linear Gaussian Systems

Question 1.1

From $\mathbf{z} = \mathbf{Ax} + \mathbf{b} + \mathbf{n}_1$ with $\mathbf{n}_1 \sim \mathcal{N}(\mathbf{0}, \frac{1}{2}\mathbf{I})$, we can immediately conclude that

$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}\left(\mathbf{z}|\mathbf{Ax} + \mathbf{b}, \frac{1}{2}\mathbf{I}\right).$$

Question 1.2

The distribution $p(\mathbf{x}|\mathbf{z})$ can be seen as the posterior distribution of the solution system $\mathbf{z} = \mathbf{Ax} + \mathbf{b} + \mathbf{n}_1$ with $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and therefore, we can conclude that $p(\mathbf{x}|\mathbf{z})$ is Gaussian with some mean vector $\mathbf{m}$ and covariance matrix $\mathbf{S}$, i.e. $p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{m}, \mathbf{S})$ and use the equations in eq. (3.37) from section 3.3 in Murphy1 to calculate the mean and covariance matrix as follows.

First, we compute the covariance matrix:

$$\mathbf{S}^{-1} = \mathbf{I} + 2\mathbf{A}^T\mathbf{A} = 3\mathbf{I} \iff \mathbf{S} = \frac{1}{3}\mathbf{I}$$

since $2\mathbf{A}^T\mathbf{A} = 2\mathbf{I}$.

For the mean vector, we have:

$$\mathbf{m} = 2\mathbf{S}\mathbf{A}^T(\mathbf{z} - \mathbf{b}) = \frac{2}{3}\mathbf{A}^T(\mathbf{z} - \mathbf{b}) = \frac{2}{3} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} z_1 - 1 \\ z_2 - 1 \end{bmatrix} = \frac{2}{3} \begin{bmatrix} z_2 - 1 \\ z_1 - 1 \end{bmatrix}.$$

Question 1.3

First we use eq. (3.38) in Murphy1 to compute $p(\mathbf{z})$:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{b}, \frac{1}{2}\mathbf{I} + \mathbf{A}\mathbf{A}^T) = \mathcal{N}\left(\mathbf{z}|\mathbf{b}, \frac{3}{2}\mathbf{I}\right)$$

since $\mathbf{A}^T\mathbf{A} = \mathbf{I}$. Using eq. (3.38) again yields

$$p(\mathbf{y}) = \mathcal{N}\left(\mathbf{y}|\mathbf{W}\mathbf{b}, \frac{1}{2}\mathbf{I} + \mathbf{W}\frac{3}{2}\mathbf{I}\mathbf{W}^T\right) = \mathcal{N}\left(\mathbf{y}|\mathbf{W}\mathbf{b}, \frac{1}{2}\mathbf{I} + \frac{3}{2}\mathbf{W}\mathbf{W}^T\right) = \mathcal{N}\left(\mathbf{y}|\begin{bmatrix} 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 3.5 & 1.5 \\ 1.5 & 2 \end{bmatrix}\right).$$

Alternatively, we could also compute the mean and covariance of $p(\mathbf{y})$ as follows

$$\begin{aligned} \mathbb{E}[\mathbf{y}] &= \mathbb{E}[\mathbf{W}\mathbf{z} + \mathbf{n}_2] \\ &= \mathbb{E}[\mathbf{W}(\mathbf{A}\mathbf{x} + \mathbf{b} + \mathbf{n}_1) + \mathbf{n}_2] \\ &= \mathbb{E}[\mathbf{W}\mathbf{A}\mathbf{x} + \mathbf{W}\mathbf{b} + \mathbf{W}\mathbf{n}_1 + \mathbf{n}_2] \\ &= \mathbf{W}\mathbf{A}\mathbb{E}[\mathbf{x}] + \mathbf{W}\mathbf{b} + \mathbf{W}\mathbb{E}[\mathbf{n}_1] + \mathbb{E}[\mathbf{n}_2] \\ &= \mathbf{W}\mathbf{A}\mathbb{E}\mathbf{0} + \mathbf{W}\mathbf{b} + \mathbf{W}\mathbf{0} + \mathbf{0} \\ &= \mathbf{W}\mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \end{aligned}$$

and

$$\begin{aligned} \text{cov}[\mathbf{y}] &= \text{cov}[\mathbf{W}\mathbf{A}\mathbf{x} + \mathbf{W}\mathbf{b} + \mathbf{W}\mathbf{n}_1 + \mathbf{n}_2] \\ &= \text{cov}[\mathbf{W}\mathbf{A}\mathbf{x}] + \text{cov}[\mathbf{W}\mathbf{b}] + \mathbf{W}\text{cov}[\mathbf{n}_1]\mathbf{W}^T + \text{cov}[\mathbf{n}_2] \\ &= \mathbf{W}\mathbf{A}\text{cov}[\mathbf{x}]\mathbf{A}^T\mathbf{W}^T + \mathbf{0} + \mathbf{W}\frac{1}{2}\mathbf{I}\mathbf{W}^T + \frac{1}{2}\mathbf{I} \\ &= \mathbf{W}\mathbf{A}\mathbf{A}^T\mathbf{W}^T + \frac{1}{2}\mathbf{W}\mathbf{W}^T + \frac{1}{2}\mathbf{I} \\ &= \begin{bmatrix} 3.5 & 1.5 \\ 1.5 & 2 \end{bmatrix}. \end{aligned}$$

```
In [4]: # matrices and vectors
A = jnp.array([[0, 1], [1, 0]])
W = jnp.array([[1, 1], [1, 0]])
b = jnp.array([1, 1])

# method 1
mu_y = W@b
Sigma_y = jnp.identity(2)/2 + (3/2)*W@W.T
```

```

print('Mean y\n', mu_y)
print('Cov y\n', Sigma_y)

# method 2
mu_y = W@b
Sigma_y = W@A@A@W.T + 0.5*W@W.T + 0.5*jnp.identity(2)
print('Mean y\n', mu_y)
print('Cov y\n', Sigma_y)

```

```

Mean y
[2 1]
Cov y
[[3.5 1.5]
 [1.5 2. ]]
Mean y
[2 1]
Cov y
[[3.5 1.5]
 [1.5 2. ]]

```

Part 2: Binary classification

```

In [5]: # data
X = jnp.array([[0, 0], [1, 1]])
y = jnp.array([0, 1])
lam = 1/4
w_MAP = 0.74077439*jnp.array([1, 1])
x_star = jnp.array([0, 1])

```

Questions 2.1

We have

$$p(y^*|y, \mathbf{x}^*) \approx \text{Ber}(y^*|\sigma(\hat{\mathbf{w}}_{\text{MAP}}^T \mathbf{x}^*)) = \text{Ber}(y^*|\sigma(0.74077439)) \approx 0.68$$

```

In [6]: sigmoid = lambda x: 1/(1+ jnp.exp(-x))
p = sigmoid(w_MAP.T@x_star)
print(f'p(y*|y, x*) = Ber(y*|p) with p = {p:3.2f}')
```

$p(y^*|y, \mathbf{x}^*) = \text{Ber}(y^*|p)$ with $p = 0.68$

Question 2.2

The quantity $\log p(\mathbf{y}, \mathbf{w})$ at the MAP estimator is given by

$$\log p(\mathbf{y}, \hat{\mathbf{w}}_{\text{MAP}}) \approx \log \mathcal{N}(\hat{\mathbf{w}}_{\text{MAP}} | \mathbf{0}, \lambda^{-1} \mathbf{I}) + \log \text{Ber}(y_1 | \sigma(\hat{\mathbf{w}}_{\text{MAP}}^T \mathbf{x}_1)) + \log \text{Ber}(y_2 | \sigma(\hat{\mathbf{w}}_{\text{MAP}}^T \mathbf{x}_2)) \approx -6.05.$$

```
In [36]: log_npdf = lambda x, m, v: -0.5*jnp.log(2*jnp.pi*v) - 0.5*(x-m)**2/v
log_ber = lambda y, p: y*jnp.log(p) + (1-y)*jnp.log(1-p)

def log_joint(w):
    p = sigmoid(X@w)
    log_prior = jnp.sum(log_npdf(w, 0, 1/lam))
    log_likelihood = jnp.sum(log_ber(y, p))
    return log_prior + log_likelihood

print(f'log p(y,w) at w_MAP: {log_joint(w_MAP):3.2f}')
```

log p(y,w) at w_MAP: -4.26

Question 2.3

Since $\hat{\mathbf{w}}_{\text{MAP}}$ is stated to be the MAP-estimator, we know that the gradient is equal to zero, i.e.

$$\nabla_{\mathbf{w}} \log p(\mathbf{y}, \mathbf{w})|_{\mathbf{w}=\hat{\mathbf{w}}_{\text{MAP}}} = \mathbf{0}.$$

From exercise 4, we know that:

$$\mathcal{H} = \nabla_{\mathbf{w}}^2 \log p(\mathbf{y}, \mathbf{w})|_{\mathbf{w}=\hat{\mathbf{w}}_{\text{MAP}}} = -\mathbf{X}^T \mathbf{S} \mathbf{X} - \lambda \mathbf{I} \approx \begin{bmatrix} -0.40 & -0.15 \\ -0.15 & -0.40 \end{bmatrix},$$

where $\mathbf{S} \in \mathbb{R}^{N \times N}$ is a diagonal matrix with $S_{nn} = (1 - \sigma(f_n))\sigma(f_n)$ and $\mathbf{X} \in \mathbb{R}^{N \times D}$ is the design matrix such that $\mathbf{x}_n$ will be the n 'th row of $\mathbf{X}$ and $f_n = \hat{\mathbf{w}}_{\text{MAP}}^T \mathbf{x}_n$.

Note that you can get the opposite sign of the Hessian depending on how you define the Taylor approximation in the Laplace approximation. Both are accepted as long as you use the correct sign in Question 2.4

```
In [38]: p = sigmoid(X@w_MAP)
S = jnp.diag((1-p)*p)
H = -X.T@S@X - lam*jnp.identity(2)

print('Hessian:\n', np.array2string(H, precision=2))
```

Hessian:
[[-0.4 -0.15]
[-0.15 -0.4]]

Question 2.4

The Laplace approximation is given by $q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\hat{\mathbf{w}}_{\text{MAP}}, \mathbf{S})$, where $\mathbf{S} = -\mathcal{H}^{-1} \approx \begin{bmatrix} 2.91 & -1.09 \\ -1.09 & 2.91 \end{bmatrix}$ (again, the sign in front of Hessian depends on how the Taylor approximation was set up). After obtaining the Laplace approximation, we can either use Monte Carlo sampling or the probit approximation to compute the posterior predictive distribution. We have

$$p(f^*|\mathbf{y}, \mathbf{x}^*) \approx \mathcal{N}(f^*|\hat{\mathbf{w}}_{\text{MAP}}^T \mathbf{x}^*, \mathbf{x}_*^T \mathbf{S} \mathbf{x}^*) \approx \mathcal{N}(f^*|0.74, 2.91)$$

Finally, using the Probit approximation yields

$$p(y^*|\mathbf{y}, \mathbf{x}^*) \approx \text{Ber}(y^*|p^*) \approx \text{Ber}(y^*|0.62)$$

since

$$\Phi\left(\frac{0.74}{\sqrt{\frac{8}{\pi} + 2.91}}\right) \approx 0.62.$$

```
In [40]: from scipy.stats import norm
covariance_matrices = { 'Hessian from Q2.3': -jnp.linalg.inv(H),
                        'Alternative Hessian from exercise text': 2/5*jnp.identity(2)}

for name, S in covariance_matrices.items():

    m = w_MAP.T@x_star
    v = x_star.T@S@x_star
    p_ystar1 = norm.cdf(m/np.sqrt(8/np.pi + v))
    print(60*'*')
    print(f'Solution using {name}')
    print(60*'*')
    print(f'Covariance matrix for Laplace\n', np.array2string(S, precision=2))
    print('')
    print(f'p(f^*|y, x^*) = N(f^*|{m:3.2f}, {v:3.2f})')
    print(f'p(y^* = 1|y, x^*) = {p_ystar1:3.2f} (probit approximation)')
```

```
*****
Solution using Hessian from Q2.3
*****
Covariance matrix for Laplace
[[ 2.91 -1.09]
 [-1.09  2.91]]

p(f^*|y, x^*) = N(f^*|0.74, 2.91)
p(y^* = 1|y, x^*) = 0.62 (probit approximation)
*****
Solution using Alternative Hessian from exercise text
*****
Covariance matrix for Laplace
[[0.4 0. ]
 [0.  0.4]]

p(f^*|y, x^*) = N(f^*|0.74, 0.40)
p(y^* = 1|y, x^*) = 0.67 (probit approximation)
```

Question 2.5

Since the new set of response variables, $z_n \in \{0, 1, 2 \dots\}$, are non-negative integers the Bernoulli likelihood is no longer appropriate. A more appropriate choice would be the Poisson distribution with the `log` link function such that

$$p(z_n|\mathbf{x}_n, \mathbf{w}) = \text{Poisson}(z_n|e^{\mathbf{w}^T \mathbf{x}}),$$

but there are also other possible choices.

Part 3: A simple mixture model

Question 3.1

Using the sum rule and product rule, we have that

$$p(y|v) = \sum_s p(y, s|v) = \sum_s p(y|v, s)p(s) = \sum_{s \in \{0,1\}} \mathcal{N}(y|v, \sigma^2 + s\tau^2) \text{Ber}(s|p) = (1-p)\mathcal{N}(y|v, \sigma^2) + p\mathcal{N}(y|v, \sigma^2 + \tau^2) = \frac{3}{4}\mathcal{N}(y|v, 1) + \frac{1}{4}\mathcal{N}(y|v, 3),$$

which is a Gaussian mixture model with two components. The mixture weights are 3/4 and 1/4, respectively, and the variances are 1 and 3, respectively. Both components have a mean equal to v .

Question 3.2

Using the sum rule, we get

$$\begin{aligned} p(y) &= \sum_{s \in \{0,1\}} \int p(y, v, s) dv \\ &= \sum_{s \in \{0,1\}} \int p(y|v, s)p(v)p(s) dv \\ &= \int \left[\sum_{s \in \{0,1\}} p(y|v, s)p(s) \right] p(v) dv \\ &= \int \left[\frac{3}{4}\mathcal{N}(y|v, 1) + \frac{1}{4}\mathcal{N}(y|v, 3) \right] \mathcal{N}(v|1, 1) dv \\ &= \frac{3}{4} \int \mathcal{N}(y|v, 1)\mathcal{N}(v|1, 1) dv + \frac{1}{4} \int \mathcal{N}(y|v, 3)\mathcal{N}(v|1, 1) dv \end{aligned}$$

Using eq. (3.38) from Murphy¹ yields:

$$p(y) = \frac{3}{4}\mathcal{N}(y|1, 2) + \frac{1}{4}\mathcal{N}(y|1, 4).$$

Question 3.3

First we compute:

$$p(y|s=1) = \int p(y|v, s=1)p(v)dv = \int \mathcal{N}(y|v, 1+2)\mathcal{N}(v|1, 1)dv = \mathcal{N}(y|1, 1+2+1) = \mathcal{N}(y|1, 4)$$

Then we use Bayes' rule to get:

$$p(s=1|y) = \frac{p(y|s=1)p(s=1)}{p(y)} = \frac{\frac{1}{4}\mathcal{N}(y|1, 4)}{\frac{3}{4}\mathcal{N}(y|1, 2) + \frac{1}{4}\mathcal{N}(y|1, 4)}$$

Notice that even if we did not solve Q3.2 first, we could still solve this because we know that $p(s=1|y) + p(s=0|y) = 1$, and the posterior $p(s=i|y)$ is proportional to the product of likelihood and prior, i.e. $p(s=i|y) = \frac{1}{k}p(y|s=i)p(s=i)$.

Combining these yields

$$1 = p(s=1|y) + p(s=0|y) = \frac{1}{k}p(y|s=1)p(s=1) + \frac{1}{k}p(y|s=0)p(s=0),$$

and hence,

$$k = p(y|s=1)p(s=1) + p(y|s=0)p(s=0).$$

Part 4: Regression models

Question 4.1

The prior probability is $p(w_1 > 0) = 0$ by symmetry (the prior is a zero-mean Gaussian, which is symmetric around zero).

The posterior distribution for w_1 is $p(w_1|\mathbf{y}) = \mathcal{N}(w_1 | -0.87, 0.41)$. Hence, the posterior standard deviation is $\sigma_1 = \sqrt{0.41}$. Therefore, the posterior probability of the event w_1 is the same as

$$p(-0.87 + \sqrt{0.41}z_1 > 0) = p\left(z_1 > \frac{0.87}{\sqrt{0.41}}\right) = 1 - p\left(z_1 \leq \frac{0.87}{\sqrt{0.41}}\right) = 1 - \Phi\left(\frac{0.87}{\sqrt{0.41}}\right) \approx 0.087 \quad (1)$$

where $z_1 \sim \mathcal{N}(0, 1)$ and $\Phi(\cdot)$ is the CDF of a standard normal distribution.

```
In [113... from scipy.stats import norm
m = -0.87
v = 0.41
print(f'Posterior prob of event w1 > 0 = {1-norm.cdf(-m/jnp.sqrt(v)):3.2f}')
```

Posterior prob of event w1 > 0 = 0.09

Let's verify using sampling:

```
In [114... w1_samples = m + jnp.sqrt(v)*random.normal(random.key(1), shape=(100000,))
print(f'Posterior prob of event w1 > 0 = {jnp.mean(w1_samples > 0):3.2f} (using samples)')
```

Posterior prob of event $w_1 > 0 = 0.09$ (using samples)

Question 4.2

We have $p(w_2|\mathbf{y}) = \mathcal{N}(w_2|2.13, 0.27)$ and therefore,

$$p(1.46 < w_2 < 2.80|\mathbf{y}) \approx 0.8.$$

```
In [123... lower, upper = norm.interval(confidence=0.8, loc=2.13, scale=jnp.sqrt(0.27))
print(f'p({lower:3.2f} < w_2 < {upper:3.2f}|\mathbf{y}) \sim 0.8')
```

$p(1.46 < w_2 < 2.80|\mathbf{y}) \sim 0.8$

Question 4.3

For $\mathbf{w} \sim \mathcal{N}(\mathbf{m}, \mathbf{S})$ and $\phi = \begin{bmatrix} 1 & x^* \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix}$, we have

$$f^* = \mathbf{w}^T \phi \sim \mathcal{N}(\mathbf{m}^T \phi, \phi^T \mathbf{S} \phi) \approx \mathcal{N}(1.26, 0.90)$$

```
In [126... m = jnp.array([-0.87, 2.13])
S = jnp.array([[0.41, 0.11], [0.11, 0.27]])
phi = jnp.array([1, 1])
print(f'p(f*|\mathbf{y}) = N(f*|{phi@m:3.2f}, {phi.T@S@phi:3.2f})')
```

$p(f^*|\mathbf{y}) = \mathcal{N}(f^*|1.26, 0.90)$

Question 4.4

We have

$$p(y^*|\mathbf{y}, x^*) = \mathcal{N}(y^*|\mathbf{m}^T \phi, \phi^T \mathbf{S} \phi + \sigma^2) \approx \mathcal{N}(y^*|1.26, 2.31)$$

```
In [127... sigma2 = jnp.sqrt(2)
print(f'p(y*|\mathbf{y}) = N(f*|{phi@m:3.2f}, {phi.T@S@phi + sigma2:3.2f})')
```

$p(y^*|\mathbf{y}) = \mathcal{N}(f^*|1.26, 2.31)$

Question 4.5

Since q is a factorized Gaussian distribution, the entropy is given by

$$\mathcal{H}[q] = \frac{1}{2} \sum_{i=1}^D \log(2\pi e v_i) \approx 2.14,$$

where $D = 4$ is the number of dimensions.

```
In [130.. m = jnp.array([-0.82, 2.37, -0.88, 0.37])
v = jnp.array([0.38, 0.19, 0.13, 0.09])
H = 0.5*jnp.sum(jnp.log(2*jnp.exp(1)*jnp.pi*v))
print(f'The entropy is: {H:3.2f}')
```

The entropy is: 2.14

Question 4.6

We can write the Monte Carlo estimator as follows:

$$p(y^*|\mathbf{y}, x^*) = \int \mathcal{N}(y^*|w_1 + w_2 x^*, e^{w_3 + w_4 x^*}) q(\mathbf{w}) d\mathbf{w} \quad (2)$$

$$= \frac{1}{S} \sum_{i=1}^S \mathcal{N}(y^*|w_1^{(i)} + w_2^{(i)} x^*, e^{w_3^{(i)} + w_4^{(i)} x^*}), \quad (3)$$

where $\mathbf{w}^{(i)} \sim q(\mathbf{w})$. Inserting $y^* = 1$ yields

$$p(1|\mathbf{y}, x^*) = \frac{1}{S} \sum_{i=1}^S \mathcal{N}(1|w_1^{(i)} + w_2^{(i)} x^*, e^{w_3^{(i)} + w_4^{(i)} x^*}) \approx 0.23 \quad (4)$$

```
In [140.. S = 10000
key = random.key(1)
w_samples = m + jnp.sqrt(v)*random.normal(key, shape=(S, 4))
npdf = lambda x, m, v: np.exp(log_npdf(x, m, v))

vals = npdf(1, w_samples[:, 0] + w_samples[:, 1], jnp.exp(w_samples[:, 2] + w_samples[:, 3]))

print(f'p(y^* = 1|y, x*) = {jnp.mean(vals):3.2f}')
```

$p(y^* = 1|\mathbf{y}, x^*) = 0.23$

Part 5

Question 5.1

```
In [158.. import sys
sys.path.append('../2025/exercises/exercise9')
from exercise9 import metropolis
x = 1
y = 2
```

```
f = lambda x, w1, w2: w2*jnp.tanh(w1*x)

def log_target(w):
    w1, w2 = w
    v = f(x, w1, w2)
    return log_npfd(y, v, 2) + log_npfd(w1, 0, 1)+log_npfd(w2, 0, 1)

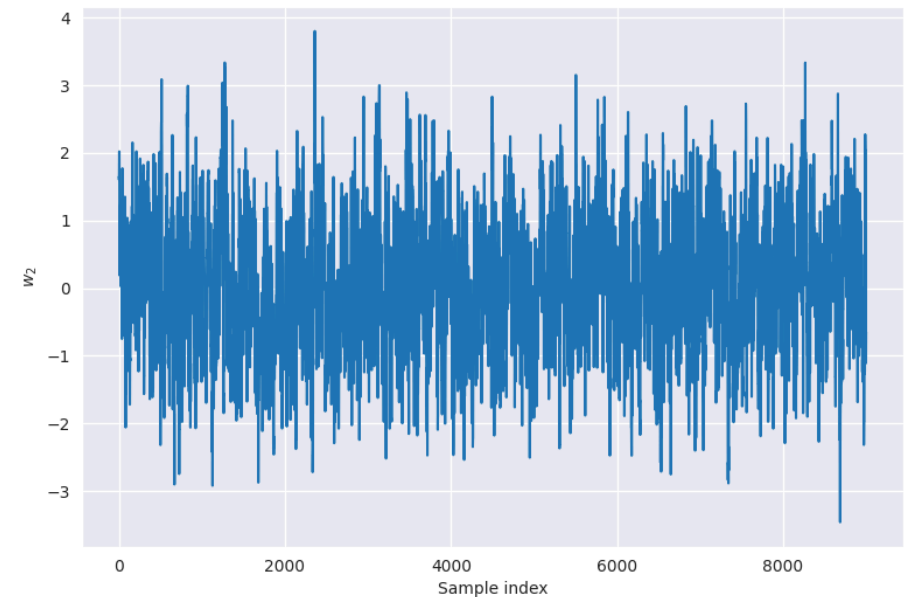
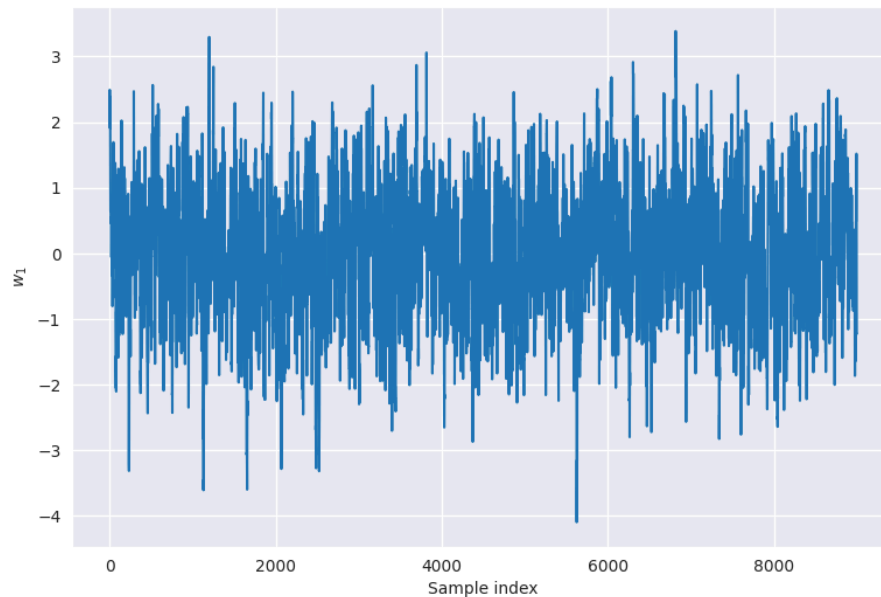
thetas, _ = metropolis(log_target, num_params=2, tau=1, num_iter=10000, theta_init=jnp.array([0, 0]))
```

Acceptance ratio: 0.54

```
In [159... thetas_after_warmup = thetas[1000:, :]
fig, ax = plt.subplots(1, 2, figsize=(20, 6))
ax[0].plot(thetas_after_warmup[:, 0])
ax[1].plot(thetas_after_warmup[:, 1])
ax[0].set(xlabel='Sample index', ylabel='$w_1$')
ax[1].set(xlabel='Sample index', ylabel='$w_2$')
fig.suptitle('Trace plots')
```

Out[159... Text(0.5, 0.98, 'Trace plots')

Trace plots



Question 5.2

We have

$$p(f^* > 1|y_1, x^* = 2) \approx \frac{1}{S} \sum_{i=1}^S \mathbb{I}[f^* > 1] = \frac{1}{S} \sum_{i=1}^S \mathbb{I}\left[w_2^{(i)} \tanh(w_1^{(i)} x^*) > 1\right] \approx 0.20$$

```
In [165... xstar = 2
w1_samples, w2_samples = thetas_after_warmup[:, 0], thetas_after_warmup[:, 1]
fstar_values = f(xstar, w1_samples, w2_samples)
print(f'p(f* > 1|y, x*) = {jnp.mean(fstar_values > 1):3.2f}')
```

p(f* > 1|y, x*) = 0.20

Question 5.3

Since $y^* = f(x^*) + \epsilon^*$, we have

```
In [175... S = len(w1_samples)
key = random.key(123)
sigma2 = 2
epsilon_values = jnp.sqrt(sigma2)*random.normal(key, shape=(S,))
ystar_values = fstar_values + epsilon_values
ystar_mean = jnp.mean(ystar_values)
ystar_lower, ystar_upper = jnp.percentile(ystar_values, jnp.array([2.5, 97.5]))
ystar_mean = jnp.mean(ystar_values)

print(f'Mean of y^*: {ystar_mean:3.2f}')
```

Mean of y^*: 0.36

95% interval for y^*: -2.77-3.55